# Unit 3

## Exception Handling

# Introduction

- Users have high expectations for the code we produce.
- Users will use our programs in unexpected ways.
- Due to design errors or coding errors, our programs may fail in unexpected ways during execution
- It is our responsibility to produce quality code that does not fail unexpectedly.
- Consequently, we must design error handling into our programs.

# Errors

- An **Error** is any unexpected result obtained from a program during execution.
- Unhandled errors may manifest themselves as incorrect results or behavior, or as abnormal program termination.
- Errors should be handled by the programmer, to prevent them from reaching the user.

# Errors

- Some typical causes of errors:
  I. Memory errors (i.e. memory incorrectly allocated, memory leaks, “null pointer”)
  II. File system errors (i.e. disk is full, disk has been removed)
  III. Network errors (i.e. network is down, URL does not exist)
  IV. Calculation errors (i.e. divide by 0)
  V. Array errors (i.e. accessing element –1)
  VI. Conversion errors (i.e. convert ‘q’ to a number)

# Error Handling

**Traditional Error Handling**

1. Every method returns a value (flag) indicating either success, failure, or some error condition. The calling method checks the return flag and takes appropriate action.

Downside: programmer must remember to always check the return value and take appropriate action. This requires much code (methods are harder to read) and something may get overlooked.

# Error Handling

**Traditional Error Handling**

2. Create a global error handling routine, and use some form of “jump” instruction to call this routine when an error occurs.

Downside: “jump” instruction (GoTo) are considered “bad programming practice” and are discouraged. Once you jump to the error routine, you cannot return to the point of origin and so must (probably) exit the program.

# Exceptions

**Exceptions** – a better error handling

Exceptions are a mechanism that provides the best of both worlds.

Exceptions act similar to method return flags in that any method may raise an exception should it encounter an error.

Exceptions act like global error methods in that the exception mechanism is built into Java; exceptions are handled at many levels in a program, locally and/or globally.

# Benefits of exception handling

- Separating Error-Handling code from “regular” business logic code
- Propagating errors up the call stack
- Grouping and differentiating error types

```
statement 1;
statement 2;
statement 3;
statement 4;
statement 5;//exception occurs
statement 6;
statement 7;
statement 8;
statement 9;
statement 10;
```

# Exception Constructs

- Five constructs are used in exception handling:

1. try – a block surrounding program statements to monitor for exceptions
2. catch – together with try, catches specific kinds of exceptions and handles them in some way
3. finally – specifies any code that absolutely must be executed whether or not an exception occurs
4. throw – used to throw a specific exception from the program
5. throws – specifies which exceptions a given method can throw

# Exception-Handling Block

General form:

```
try { … }
catch(Exception1 ex1) { … }
catch(Exception2 ex2) { … }
…
finally { … }
```

where:

1) try { … } is the block of code to monitor for exceptions
2) catch(Exception ex) { … } is exception handler for the exception Exception
3) finally { … } is the block of code to execute before the try block ends

# Grouping and Differentiating Error Types

- Because all exceptions thrown within a program are objects, the grouping or categorizing of exceptions is a natural outcome of the class hierarchy.
- An example of a group of related exception classes in the Java platform are those defined in java.io.IOException and its descendants.
- IOException is the most general and represents any type of error that can occur when performing I/O.
- Its descendants represent more specific errors. For example, FileNotFoundException means that a file could not be located on disk.

# Grouping and Differentiating Error Types

- A method can write specific handlers that can handle a very specific exception.
- The FileNotFoundException class has no descendants, so the following handler can handle only one type of exception.

```
catch (FileNotFoundException e) {
...
}
```

- A method can catch an exception based on its group or general type by specifying any of the exception's super classes in the catch statement.
- For example, to catch all I/O exceptions, regardless of their specific type, an exception handler specifies an IOException argument.

```
// Catch all I/O exceptions, including
// FileNotFoundException, EOFException, and so on.
catch (IOException e) {
...
}
```

# Termination vs. Resumption

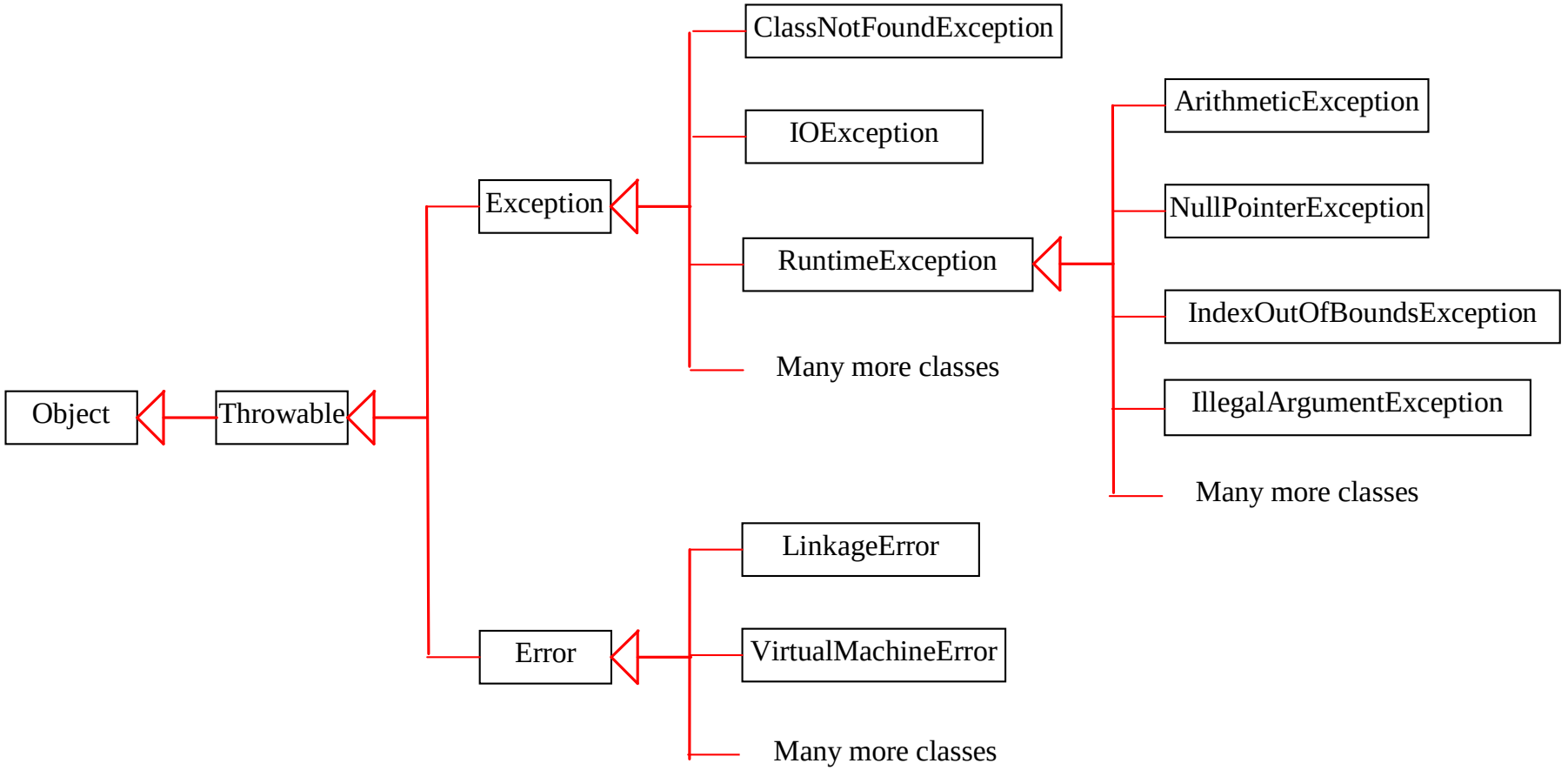
- There are two basic models in exception-handling theory.
- In termination the error is so critical there's no way to get back to where the exception occurred. Whoever threw the exception decided that there was no way to salvage the situation, and they don't want to come back.
- The alternative is called resumption. It means that the exception handler is expected to do something to rectify the situation, and then the faulting method is re-tried, presuming success the second time. If you want resumption, it means you still hope to continue execution after the exception is handled.

# Termination vs. Resumption

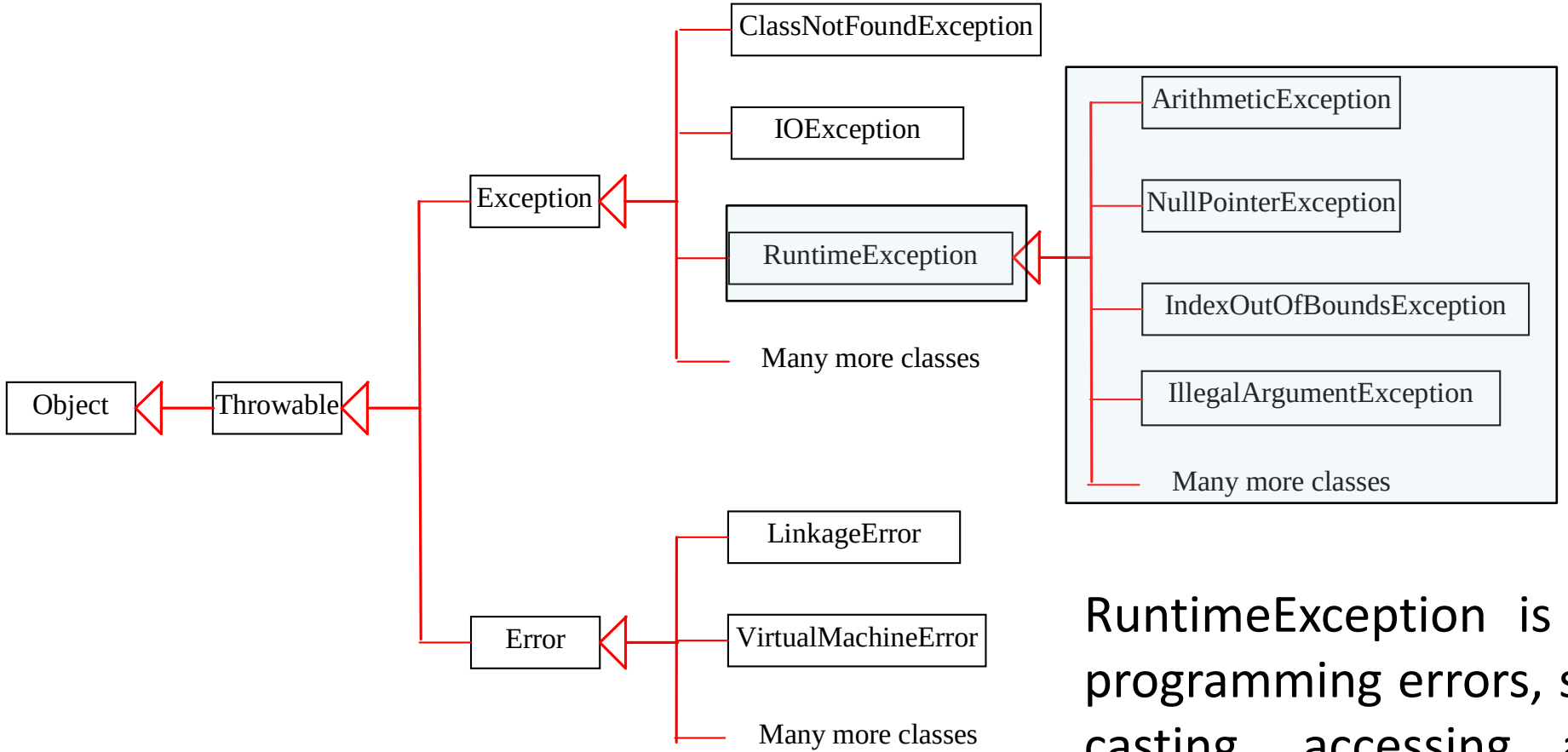
- In resumption a method call that wants resumption-like behavior (i.e don't throw an exception, call a method that fixes the problem.)
- Alternatively, place your try block inside a while loop that keeps reentering the try block until the result is satisfactory.
- Operating systems that supported resumptive exception handling eventually ended up using termination-like code and skipping resumption.

# Exception Types

- All exceptions are sub-classes of the built-in class Throwable.
- Throwable contains two immediate sub-classes:

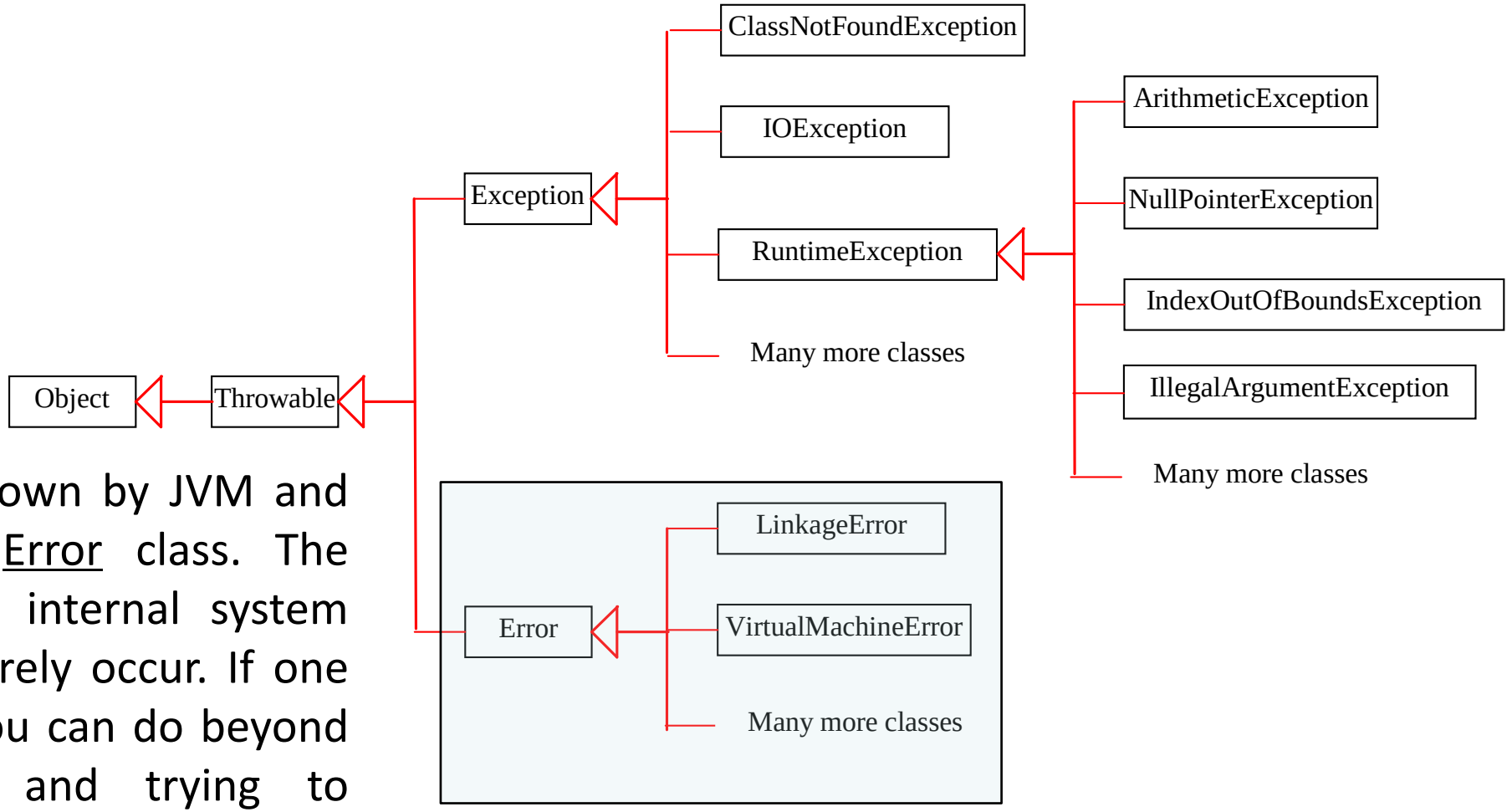


# Runtime Exceptions




RuntimeException is caused by programming errors, such as bad casting, accessing an out-of-bounds array, and numeric errors.

# System Errors




*System errors* are thrown by JVM and represented in the Error class. The Error class describes internal system errors. Such errors rarely occur. If one does, there is little you can do beyond notifying the user and trying to terminate the program gracefully.

# Checked Exceptions vs. Unchecked Exceptions

RuntimeException, Error and their subclasses are known as *unchecked exceptions*. All other exceptions are known as *checked exceptions*, meaning that the compiler forces the programmer to check and deal with the exceptions.

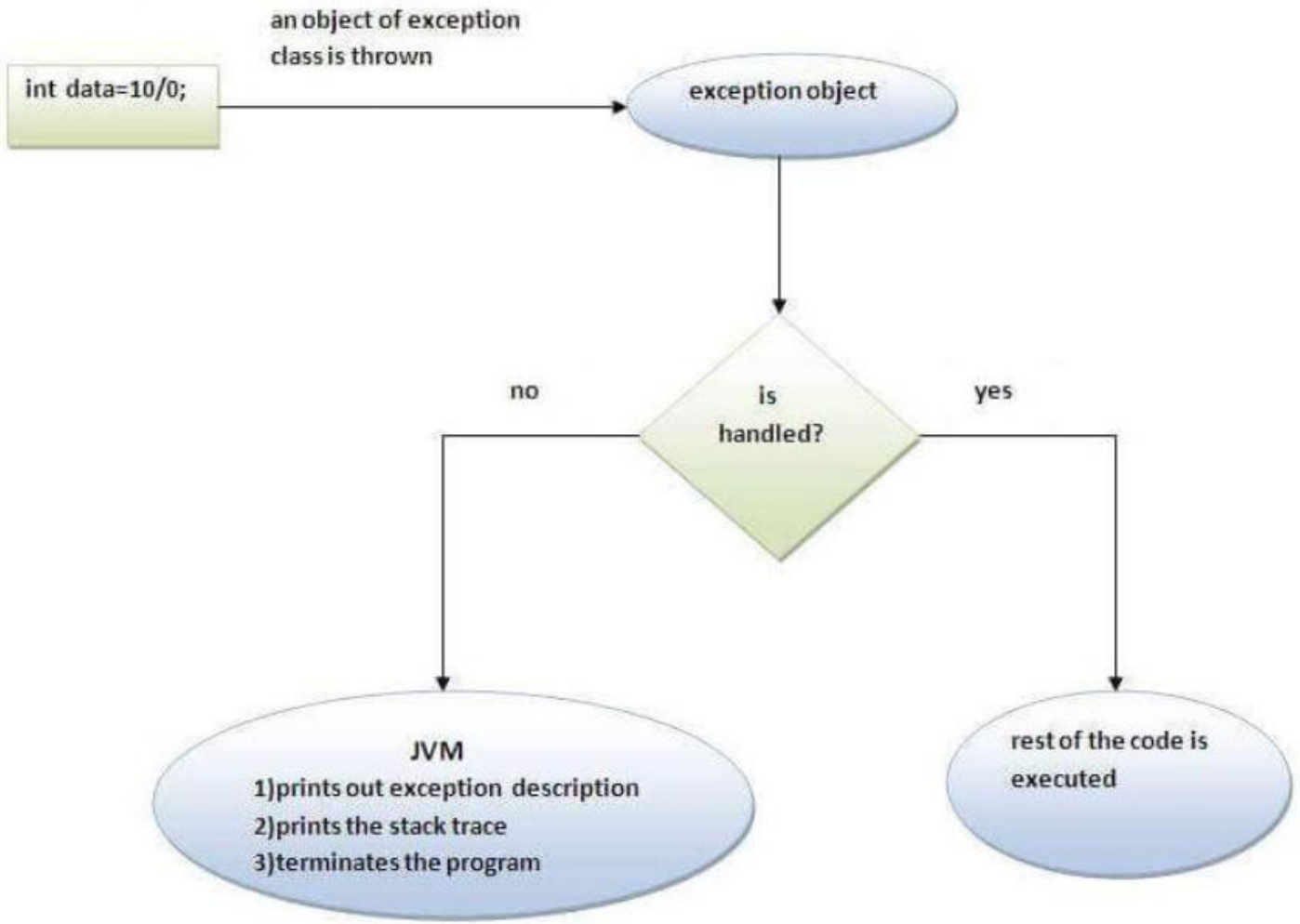
# Exception Handling Example

```
public class JavaExceptionExample{
  public static void main(String args[]){
   try{
      //code that may raise exception
      int data=100/0;
   }catch(ArithmeticException e){System.out.println(e);}
   //rest code of the program
   System.out.println("rest of the code...");
  }
}
```

Output:

```
Exception in thread main java.lang.ArithmeticException:/ by zero
rest of the code...
```

# Internal working of java try-catch block



# Catching Exceptions : Multiple

```
try {
  statements;   // Statements that may throw
  exceptions
}
catch (Exception1 exVar1) {
  handler for exception1;
}
catch (Exception2 exVar2) {
  handler for exception2;
}
...
catch (ExceptionN exVar3) {
  handler for exceptionN;
}
```

```
public static void main(String[] args) {

    try{
        int a[]=new int[5];
        a[5]=30/0;
       }
       catch(ArithmeticException e)
         {
          System.out.println("Arithmetic Exception occurs");
         }
       catch(ArrayIndexOutOfBoundsException e)
         {
          System.out.println("ArrayIndexOutOfBounds Exception occurs");
         }
       catch(Exception e)
         {
          System.out.println("Parent Exception occurs");
         }
       System.out.println("rest of the code");
}
```

At a time only one exception occurs and at a time only one catch block is executed.

All catch blocks must be ordered from most specific to most general, i.e. catch for ArithmeticException must come before catch for Exception

# Catching Exceptions: Nested try's

```
class NestedTryDemo1 {
    public static void main(String args[]){
        try {
            int a = Integer.parseInt(args[0]);
            try {
                int b = Integer.parseInt(args[1]);
                System.out.println(a/b);
            } catch (ArithmeticException e) {
                System.out.println(“Div by zero error!");
            }
        } catch (ArrayIndexOutOfBoundsException) {
            System.out.println(“Need 2 parameters!");
        }
    }
}
```

# Catching Exceptions: Nested try's with methods

```
class NestedTryDemo2 {
      static void nestedTry(String args[]) {
            try {
                  int a = Integer.parseInt(args[0]);
                  int b = Integer.parseInt(args[1]);
                  System.out.println(a/b);
            } catch (ArithmeticException e) {
                  System.out.println("Div by zero error!");
            }
       }
public static void main(String args[]){
      try {
            nestedTry(args);
      } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Need 2 parameters!");
      }
 } }
```

# Unhandled Exceptions

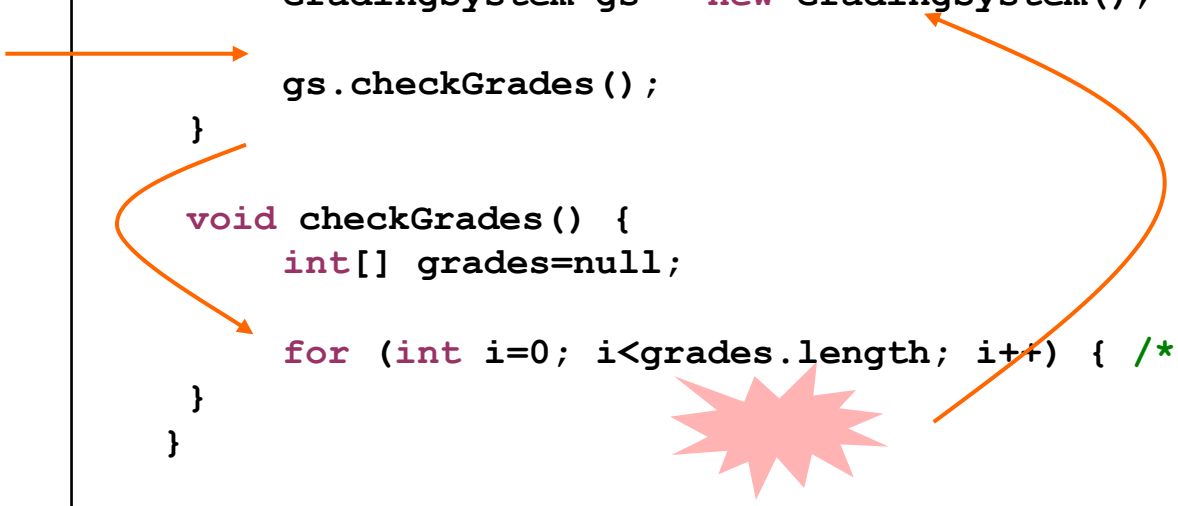
```
public class GradingSystem {

 public static void main(String[] args) {
      GradingSystem gs = new GradingSystem();

      gs.checkGrades();
 }

 void checkGrades() {
      int[] grades=null;

      for (int i=0; i<grades.length; i++) { /* test here*/ };
 }
}
```



```
Exception in thread "main" java.lang.NullPointerException
 at codesnippets.GradingSystem.checkGrades(GradingSystem.java:31)
 at codesnippets.GradingSystem.main(GradingSystem.java:6)
```

# Handling Exceptions

```
public class GradingSystem {

 public static void main(String[] args) {
      GradingSystem gs = new GradingSystem();

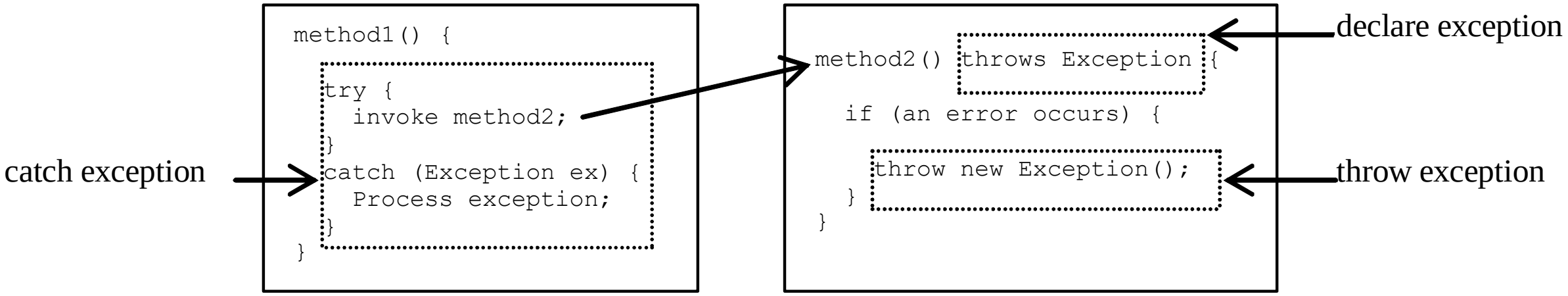
      gs.checkGrades();
 }

 void checkGrades() {
      int[] grades=null;

      try {
           for (int i=0; i<grades.length; i++) { /* test here*/ };
      } catch (NullPointerException e) {
           System.out.println("Grades may be empty!");
      } catch (RuntimeException e) {
           System.out.println("Problem while executing!");
      } catch (Exception e) {
           System.out.println("Error in checking grades!");
      } finally {
           System.out.println("Finished checking grades.");
      }
 }
}
```

```
Grades may be empty!
Finished checking
```

# Declaring, Throwing, and Catching Exceptions



# Declaring Exceptions

Every method must state the types of checked exceptions it might throw. This is known as *declaring exceptions*.

```
public void myMethod()
   throws IOException

public void myMethod()
   throws IOException, OtherException
```

# throws Declaration

• If a method is capable of causing an exception that it does not handle, it must specify this behavior by the throws clause in its declaration:

```
type name(parameter-list) throws exception-list {
    …
}
```

•where exception-list is a comma-separated list of all types of exceptions that a method might throw.

•All exceptions must be listed except Error and RuntimeException or any of their subclasses, otherwise a compile-time error occurs.

# Throwing Exceptions

When the program detects an error, the program can create an instance of an appropriate exception type and throw it. This is known as *throwing an exception*. Here is an example,

```
throw new TheException();
```

```
TheException ex = new TheException();
throw ex;
```

# Throwing Exceptions Example

```
   /** Set a new radius */
public void setRadius(double newRadius)
    throws IllegalArgumentException {
  if (newRadius >= 0)
    radius =  newRadius;
  else
    throw new IllegalArgumentException(
      "Radius cannot be negative");
}
```

# Throwing Exceptions(throw)

- ThrowableInstance must be an object of type Throwable or its subclass.

Once an exception is thrown by:

throw ThrowableInstance;

1. the flow of control stops immediately.
2. the nearest enclosing try statement is inspected if it has a catch statement that matches the type of exception:
3. if one exists, control is transferred to that statement
4. otherwise, the next enclosing try statement is examined
5. if no enclosing try statement has a corresponding catch clause, the default exception handler halts the program and prints the stack

# Creating Exceptions

**Two ways to obtain a Throwable instance:**

1) creating one with the **new** operator

All Java built-in exceptions have at least two Constructors:

One without parameters and another with one String parameter:

```
throw new NullPointerException("demo");
```

```
public class ThrowableExample {
    public static void main(String[] args) {
        try {
            throw new NullPointerException("This is a demo exception");
        } catch (NullPointerException e) {
            System.out.println("Caught Exception: " + e.getMessage());
        }
    }
}
```

## 2) using a **parameter of the catch** clause

try { … } catch(Throwable e) { … e … }

```
public class ThrowableExample {
    public static void main(String[] args) {
        try {
            int result = 10 / 0; // This will cause an ArithmeticException
        } catch (Throwable e) {
            System.out.println("Caught Throwable: " + e);
        }
    }
}
```

# Creating Own Exception Classes

- Built-in exception classes handle some generic errors.
- For application-specific errors one can define their own exception classes.

How? Define a subclass of Exception:

```
class MyException extends Exception { … }
```

- MyException need not implement anything – its mere existence in the type system allows to use its objects as exceptions.

# Custom Exception Class

```
//
public class InsufficientFundsException extends Exception {
    private double amount;

    public InsufficientFundsException(double amount) {
        super("Insufficient funds for withdrawal. Shortfall: " + amount);
        this.amount = amount;
    }

    public double getAmount() {
        return amount;
    }
}
```

```
public void withdraw(double amount) throws
InsufficientFundsException {
        if (amount > balance) {
            double shortfall = amount - balance;
            throw new
InsufficientFundsException(shortfall);  //
Throwing custom exception
        }
        balance -= amount;
        System.out.println("Withdrawal
successful! Remaining balance: $" + balance);
    }
public double getAmount() {
        return amount;
    }
```

```
 public static void main(String[] args) {
        BankAccount account = new BankAccount(500);
// Creating an account with $500

        try {
            account.withdraw(700);  // Attempting to
withdraw more than the balance
        } catch (InsufficientFundsException e) {
            System.out.println("Exception: " +
e.getMessage());
            System.out.println("You are short by: $" +
e.getAmount());
        }
    }
```

Exception: Insufficient funds for withdrawal. Shortfall: 200.0
You are short by: $200.0

# throw and throws

```
void validate() throws Exception3
{
 try {
      // throw an exception
      throw new Exception3();

 } catch (Exception1 ex) {
      /*
       * handle Exception1 here
       */
 } catch (Exception2 ex) {
      /*
       * handle Exception2 here
       */
 } catch (Exception3 ex) {
      /*
       * can't handle Exception3,
       * propagate to call stack
       */
      throw new Exception3(ex);
 }
}
```

`throws` clause declares the unhandled exceptions that a method can throw. It propagates the exception to the calling method. It becomes part of the method public interface

Any code that calls this method must handle the exception declared in the `throws` clause

`throw` statement explicitly throws an exception

`throw` statement propagates (rethrows) a caught exception

# throw and throws - Differences

| No. | throw | throws |
|---|---|---|
| 1) | Java throw keyword is used to explicitly throw an exception. | Java throws keyword is used to declare an exception. |
| 2) | Checked exception cannot be propagated using throw only. | Checked exception can be propagated with throws. |
| 3) | Throw is followed by an instance. | Throws is followed by class. |
| 4) | Throw is used within the method. | Throws is used with the method signature. |
| 5) | You cannot throw multiple exceptions. | You can declare multiple exceptions e.g.<br>public void method()throws IOException,SQLException. |

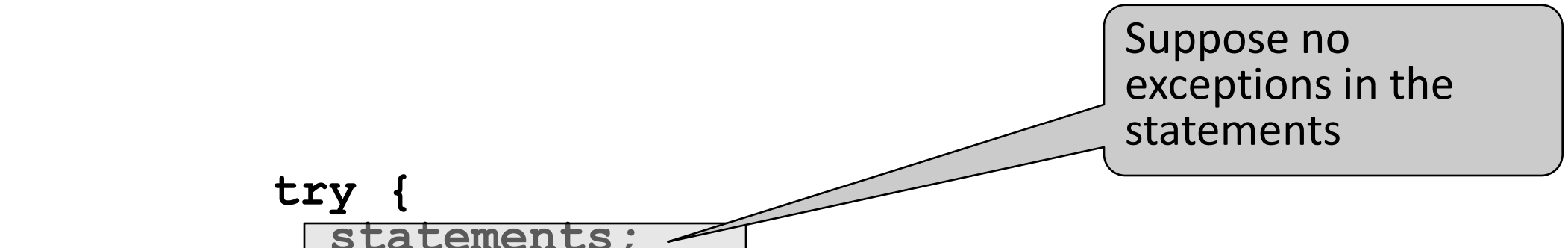
# Propagating Exceptions

```
public class GradingSystem {
 public static void main(String[] args) {
      GradingSystem gs = new GradingSystem();
      try {
           gs.checkGrades();
      } catch (Exception e) {  // must handle the exception thrown
           System.out.println(e.getMessage());
      }
 }
 void checkGrades() throws Exception {
      int[] grades={81,0,75};
      try {
           for (int i=0; i<grades.length; i++) {
                if (grades[i]<=0)
                     throw new Exception("Invalid grade!"); }
      } catch (NullPointerException e) {
           System.out.println("Grades may be empty!");
      } catch (RuntimeException e) {
           System.out.println("Problem while executing!");
      } catch (Exception e) {
           System.out.println("Can't handle error here! Rethrowing...");
           throw new Exception(e);
      }
}}
```

```
Can't handle error here!
Rethrowing...
```

animation

# Trace a Program Execution




```
try {
  statements;
}
catch(TheException ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

animation

# Trace a Program Execution

```
try {
   statements;
}
catch(TheException ex) {
   handling ex;
}
finally {
   finalStatements;
}

Next statement;
```

The final block is always executed

animation

# Trace a Program Execution

```
try {
  statements;
}
catch(TheException ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

Next statement in the method is executed

animation

# Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

Suppose an exception of type Exception1 is thrown in statement2

animation

# Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

The exception is handled.

animation

# Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

The final block is always executed.

animation

# Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

The next statement in the method is now executed.

animation

# Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1
  ex) {
  handling ex;
}
catch(Exception2
  ex) {
  handling ex;
  throw ex;
}
finally {
```

statement2 throws an exception of type Exception2.

animation

# Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1
  ex) {
  handling ex;
}
catch(Exception2
  ex) {
  handling ex;
  throw ex;
}
finally {
```

Handling exception

animation

# Trace a Program Execution

```
try {
   statement1;
   statement2;
   statement3;
}
catch(Exception1
  ex) {
   handling ex;
}
catch(Exception2
  ex) {
   handling ex;
   throw ex;
}
finally {
```

Execute the final block

animation

# Trace a Program Execution

```
try {
   statement1;
   statement2;
   statement3;
}
catch(Exception1
  ex) {
   handling ex;
}
catch(Exception2
  ex) {
   handling ex;
   throw ex;
}
finally {
```

Rethrow the exception and control is transferred to the caller

# Java Built-In Exceptions

- The default java.lang package provides several exception classes, all sub-classing the RuntimeException class.
- Two sets of built-in exception classes:

1.unchecked exceptions – the compiler does not check if a method handles or throws there exceptions

2.checked exceptions – must be included in the method's throws clause if the method generates but does not handle them

# Unchecked Built-In Exceptions

Methods that generate but do not handle those exceptions need not declare them in the throws clause:

1) ArithmeticException
2) ArrayIndexOutOfBoundsException
3) ArrayStoreException
4) ClassCastException
5) IllegalStateException
6) IllegalMonitorStateException
7) IllegalArgumentException
8. StringIndexOutOfBounds
9. UnsupportedOperationException
10. SecurityException
11. NumberFormatException
12. NullPointerException
13. NegativeArraySizeException
14. IndexOutOfBoundsException
15. IllegalThreadStateException

# Checked Built-In Exceptions

Methods that generate but do not handle those exceptions must declare them in the throws clause:

1. NoSuchMethodException NoSuchFieldException
2. InterruptedException
3. InstantiationException
4. IllegalAccessException
5. CloneNotSupportedException
6. ClassNotFoundException

# Cautions When Using Exceptions

- Exception handling separates error-handling code from normal programming tasks, thus making programs easier to read and to modify. Be aware, however, that exception handling usually requires more time and resources because it requires instantiating a new exception object, rolling back the call stack, and propagating the errors to the calling methods.

# When to Throw Exceptions

- An exception occurs in a method. If you want the exception to be processed by its caller, you should create an exception object and throw it. If you can handle the exception in the method where it occurs, there is no need to throw it.

**Handling the Exception in the Method (Sufficient Balance Check)**

If the system detects that a user doesn’t have enough balance for the transfer, it can handle the error directly by informing the user without throwing the exception back to the caller.

```
public void transferMoney(double amount) {
    double currentBalance = getAccountBalance();

    if (amount > currentBalance) {
        System.out.println("Transfer failed: Insufficient balance.");
        return;
    }

    // Proceed with the transfer
    currentBalance -= amount;
    System.out.println("Transfer successful. Remaining balance: " + currentBalance);
}
```

**Throwing the Exception to the Caller (For System-wide Handling)**

However, if your application requires broader exception handling, for example, logging every transaction failure or doing further auditing, you would throw the exception back to the caller, which could be a higher-level transaction manager.

In this version, if there's an issue with the balance, the method throws an InsufficientBalanceException back to the caller (processTransaction). The caller then handles the exception by logging it and informing the user that the transaction failed.

```
public void transferMoney(double amount) throws InsufficientBalanceException {
    double currentBalance = getAccountBalance();

    if (amount > currentBalance) {
        throw new InsufficientBalanceException("Insufficient balance for transfer.");
    }

    // Proceed with the transfer
    currentBalance -= amount;
    System.out.println("Transfer successful.");
}

public void processTransaction() {
    try {
        transferMoney(5000);
    } catch (InsufficientBalanceException e) {
        // Log the issue and inform the user
        System.out.println("Transaction failed: " + e.getMessage());
        logError(e);
    }
}
```

# When to Use Exceptions

When should you use the try-catch block in the code?

You should use it to deal with unexpected error conditions. Do not use it to deal with simple, expected situations. For example, the following code

```
try {
  System.out.println(refVar.toString());
}
catch (NullPointerException ex) {
  System.out.println("refVar is null");
}
```

# When to Use Exceptions

is better to be replaced by

```
if (refVar != null)
  System.out.println(refVar.toString());
else
  System.out.println("refVar is null");
```

# try-catch()-finally

```
try {
  /*
   * some codes to test here
   */
} catch (Exception1 ex) {
  /*
   * handle Exception1 here
   */
} catch (Exception2 ex) {
  /*
   * handle Exception2 here
   */
} catch (Exceptio
  /*
   * handle Exception3 here
   */
} finally {
  /*
   * always execute codes here
   */
}
```

`try` block encloses the context where a possible exception can be thrown

each `catch()` block is an exception handler and can appear several times

*Exception1* should not shadow *Exception2* which in turn should not shadow *Exception3* (based on the exception hierarchy)

`finally` block is always executed before exiting the `try` statement. `finally` block is optional but must appear once after the `catch()` blocks

at least one `catch()` block or `finally` block must appear in the `try` statement